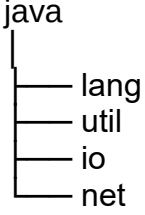
Packages and Interfaces

Java Packages

A **package** in Java is a **folder (directory)** that contains related Java classes, interfaces, enums, and sub-packages.



Each package contains related classes.

Package	Contains
java.lang	String, Math, System, Object
java.util	Scanner, ArrayList, HashMap, Collections
java.io	File, BufferedReader, FileWriter
java.net	URL, Socket

Advantages of Packages

1. Organizes the Code
2. Avoids Class Name Conflicts

3. Provides Access Protection

4. Reusability

Types of Packages

There are two types**.**

1. Built-in Packages

These are already available in Java.

java.lang

java.util

java.io

java.sql

java.net

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your name:");
        String name = sc.nextLine();
        System.out.println(name);
    }
}
```

import java.util.*; //The * means "all classes in this package."

User-defined Package

You can create your own package.

```
package mypackage;
public class Student {
    public void display() {
        System.out.println("Student Class");
    }
}
```

```
import mypackage.Student;
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Packages and Access Specifiers

```
package college.student;

public class Student {
    public int a = 10;
    protected int b = 20;
    int c = 30;
    private int d = 40;
}
```

```
package college.faculty;

import college.student.Student;

public class Faculty {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.a);
    }
}
```

Java Interface

An **interface** in Java is a blueprint of a class that specifies **what a class must do**, but not **how it does it**.

```
interface Animal {
    void sound(); //public static void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        Dog d = new Dog();
        d.sound();
    }
}
```
Dog Barks

Multiple Classes Implementing One Interface

```
interface Animal {
    void sound();
}
class Dog implements Animal {
    public void sound() {
        System.out.println("Dog Barks");
    }
}
class Cat implements Animal {
    public void sound() {
        System.out.println("Cat Meows");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();
        d.sound();
        c.sound();
    }
}
```
Dog Barks
Cat Meows

Multiple Inheritance Using Interfaces

Java **does not support multiple inheritances with classes**.

This is illegal:

```
class A {
}
class B {
}
class C extends A, B {
} // Compilation Error.
```

But Java supports multiple inheritances through interfaces

```
interface A {
    void display();
}
interface B {
```

```
    void show();
}
class C implements A, B {
    public void display() {
        System.out.println("Display");
    }
    public void show() {
        System.out.println("Show");
    }
}
public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.display();
        obj.show();
    }
}
```

Display
Show

Interface Variables

All variables in an interface are automatically:

- public
- static
- final

```
interface Demo {
    int VALUE = 100;
}
```

Java treats it as:

```
public static final int VALUE = 100;
```

```
interface Demo {
    int VALUE = 100;
}
public class Main {
    public static void main(String[] args) {
        System.out.println(Demo.VALUE);
    }
}
```

100

```
Demo.VALUE = 200; //Error because it is final.
```